Midas LZ and Axelar Integration Audit

Côme du Crest

2025-11-15

Table of contents

- Table of contents
- Midas LZ and Axelar Integration Audit
  - Scope
  - Context
  - Status
  - Legal Information And Disclaimer
- Issues
  - [Info] Trimming of sent amount may break slippage
  - Optimisations and miscellaneous

Midas LZ and Axelar Integration Audit

This document presents the findings of a smart contract audit conducted by Côme du Crest.

For any inquiry contact me by email at come.ducrest@gmail.com or via telegram @ComeduCrest.

Scope

The scope includes selected contracts within `midas-apps/contracts/contracts/misc` as of commit `0x98da740`.

The audited contracts include:

- `MidasAxelarVaultExecutable`
- `IMidasAxelarVaultExecutable`
- `MidasLzOFT`
- `MidasLzOFTAdapter`
- `MidasLzMintBurnOFTAdapter`
- `MidasLzVaultComposerSync`
- `IMidasLzVaultComposerSync`

The code has been reviewed with minor fixes as of commit `0x7d215c1`.

Context

The goal of the audited contracts is to enable cross-chain mToken bridging as well as composite operations like bridge and deposit/redeem and even bridge back received tokens. Since mTokens can grant a minter / burner role, the tokens can be bridge natively (burnt on one chain and minted on the other).

Status

The report has been sent to the core developer.

No serious issues have been found.

Info and comments have been fixed in commit `0x7d215c1`.

Legal Information And Disclaimer

1. This report is based solely on the information provided by Midas Protocol Ltd. (the “Company”), with the assumption that the information provided is authentic, accurate, complete, and not misleading as of the date of this report. The auditor has not conducted any independent enquiries, investigations or due diligence in respect of the Company, its business or its operations.
2. Changes to the information contained in the documents, repositories and any other materials referenced in this report might affect or change the analysis and conclusions presented. The auditor is not responsible for monitoring, nor will we be aware of, any future additions, modifications, or deletions to the audited code. As such, the auditor does not assume any responsibility to update any information, content or data contained in this report following the date of its publication.
3. This report does not address, nor should it be interpreted as addressing, any regulatory, tax or legal matters, including but not limited to: tax treatment, tax consequences, levies, duties, data privacy, data protection laws, issues relating to the licensing of information technology, intellectual property, money laundering and countering the financing of terrorism, or any other legal restrictions or prohibitions. The auditor disclaims any liability for omissions or errors in the findings or conclusions presented in this report.
4. The views expressed in this report are solely my views regarding the specific issues discussed within this report. This report is not intended to be exhaustive, nor should it be construed as an assurance, guarantee or warranty that the code is free from bugs, vulnerabilities, defects or deficiencies. Different use cases may carry different risks, and integration with third-party applications may introduce additional risks.
5. This report is provided for informational purposes only and should not be used as the basis for making investment or financial decisions. This report does not constitute investment research and should not be viewed as an invitation, recommendation, solicitation or offer to subscribe for or purchase any securities, investments, products or services. The auditor is not a financial advisor, and this report does not constitute financial or investment advice.
6. The statements in this report should be considered as a whole, and no individual statement should be extracted or referenced independently.
7. To the fullest extent permitted by applicable laws, the auditor disclaims any and all other liability, whether in contract, tort, or otherwise, that may arise from this report or the use thereof.

Issues

[Info] Trimming of sent amount may break slippage

Summary

In `MidasLzVaultComposerSync` when tokens are redeemed in `_redeemAndSend()` or deposited in `_depositAndSend()`, the parameter `_sendParam.minAmountLD` is used to enforce the min amount of tokens to receive from the vault operation.

The parameter `_sendParam.minAmountLD` is then set to `0` to call `_sendOft()` to send the token to the other chain if requested. Due to trimming of the sent amount to shared decimals in `OFTCore`, the actual received amount could be lower than the previously enforced `_sendParam.minAmountLD`.

Vulnerability Detail

The function `_depositAndSend()` calls `_deposit()` with `_sendParam.minAmountLD` as `_minReceiveAmount` before setting `_sendParam.minAmountLD` to zero:

```
    function _depositAndSend(
        bytes32 _depositor,
        uint256 _paymentTokenAmount,
        bytes memory _extraOptions,
        SendParam memory _sendParam,
        address _refundAddress
    ) internal {
        bool receiveToSameNetwork = _sendParam.dstEid == vaultsEid;

        bytes32 referrerId = _parseDepositExtraOptions(_extraOptions);

        uint256 mTokenAmount = _deposit(
            receiveToSameNetwork
                ? _sendParam.to.bytes32ToAddress()
                : address(this),
            _paymentTokenAmount,
            _sendParam.minAmountLD,
            referrerId
        );

        if (!receiveToSameNetwork) {
            _sendParam.amountLD = mTokenAmount;
            _sendParam.minAmountLD = 0;

            _sendOft(mTokenOft, _sendParam, _refundAddress);
        } else {
            ...
        }
```

```
        ...
    }
```

The function `_sendOft()` calls `_oft.send()` with the given `_sendParam`:

```
    function _sendOft(
        address _oft,
        SendParam memory _sendParam,
        address _refundAddress
    ) internal {
        // solhint-disable-next-line check-send-result
        IOFT(_oft).send{value: msg.value}(
            _sendParam,
            MessagingFee(msg.value, 0),
            _refundAddress
        );
    }
```

The function `oft.send()` computes the amount sent/received by removing tokens that would be lost in conversion before enforcing slippage:

```
    function _debitView(
        uint256 _amountLD,
        uint256 _minAmountLD,
        uint32 /*_dstEid*/
    ) internal view virtual returns (uint256 amountSentLD, uint256
        amountReceivedLD) {
        // @dev Remove the dust so nothing is lost on the conversion between
            chains with different decimals for the token.
        amountSentLD = _removeDust(_amountLD);
        // @dev The amount to send is the same as amount received in the
            default implementation.
        amountReceivedLD = amountSentLD;

        // @dev Check for slippage.
        if (amountReceivedLD < _minAmountLD) {
            revert SlippageExceeded(amountReceivedLD, _minAmountLD);
        }
    }

    function _removeDust(uint256 _amountLD) internal view virtual returns (
        uint256 amountLD) {
        return (_amountLD / decimalConversionRate) * decimalConversionRate;
    }
```

The issue is the same for `_redeemAndSend()`

Impact

The OFT shared decimals is provided in constructor of `MidasLzOFT` and the local decimals is set to 18. If shared decimals is 9, the conversion rate will be `1e9` and 9 decimals of the token will be

truncated.

As a result, a user could attempt to send tokens with a slippage parameter of `1e18 + 1` tokens and received only `1e18` tokens which could break composability.

Code Snippets

https://github.com/midas-apps/contracts/blob/98da740f0c58faef2551c2d5dbf01069b5455135/contracts/misc/layerzero/MidasLzVaultComposerSync.sol#L349

https://github.com/midas-apps/contracts/blob/98da740f0c58faef2551c2d5dbf01069b5455135/contracts/misc/layerzero/MidasLzVaultComposerSync.sol#L396

https://github.com/LayerZero-Labs/devtools/blob/ca5a1f9f5af52f819f727dbafbb7bff604a07c62/packages/oft-evm/contracts/OFTCore.sol#L380

https://github.com/LayerZero-Labs/devtools/blob/ca5a1f9f5af52f819f727dbafbb7bff604a07c62/packages/oft-evm/contracts/OFTCore.sol#L86

Recommendation

Truncate and round up the value `_sendParam.minAmountLD` to shared decimals before using it as a slippage parameter in `_depositAndSend()` and `_redeemAndSend()`.

Response

Fixed in commit `0x7d215c1` by not setting the `minAmountLD` value to zero.

Optimisations and miscellaneous

This part lists minor gas/code optimizations that shouldn't make the code less readable or improve overall readability. It also lists questions about unclear code segments.

Unused variables could be removed

The variables `_ONE` and `_ONE_HUNDRED_PERCENT` are unused and could be removed in both `MidasAxelarVaultExecutable` and `MidasLzVaultComposerSync`

https://github.com/midas-apps/contracts/blob/98da740f0c58faef2551c2d5dbf01069b5455135/contracts/misc/axelar/MidasAxelarVaultExecutable.sol#L84-L88

https://github.com/midas-apps/contracts/blob/98da740f0c58faef2551c2d5dbf01069b5455135/contracts/misc/layerzero/MidasLzVaultComposerSync.sol#L98-L102

Removed in commit `0x7d215c1`.

vaultsEid could be renamed

`MidasLzVaultComposerSync.vaultsEid` could be renamed to `thisChaindEid` as it represents the endpoint id for LayerZero on this chain and is not related to the specific vault.

Renamed in commit `0x7d215c1`.